

Algoritmi e Strutture Dati–parte I

lez. 5

Francesco Camastra

francesco.camastra@uniparthenope.it

Docente

- Prof. Francesco Camastra

Orario lezioni

- Mercoledì 11.00-13.00
(aula 11, Secondo Piano)
- Giovedì 9.00-11.00 (aula 11, Secondo Piano)

Ricevimento

- Orario: Mercoledì 14.00-18.00
- Dove: Stanza 430, Quarto piano, Centro Direzionale, Isola C4
- Per il ricevimento è **OBBLIGATORIO** inviare una email di prenotazione almeno due giorni prima del giorno di ricevimento.
- email del Docente:
francesco.camastra@uniparthenope.it
- Pagina del Docente:
dist.uniparthenope.it/francesco.camastra

Prerequisiti

- Programmazione I
- Matematica I
- Programmazione II
- **NOTA BENE:** Le propedeuticità non sono obbligatorie ... Ossia potreste anche sostenere l'esame senza aver dato Programmazione I e Programmazione II

Frequenza

- La Frequenza del corso non è obbligatoria

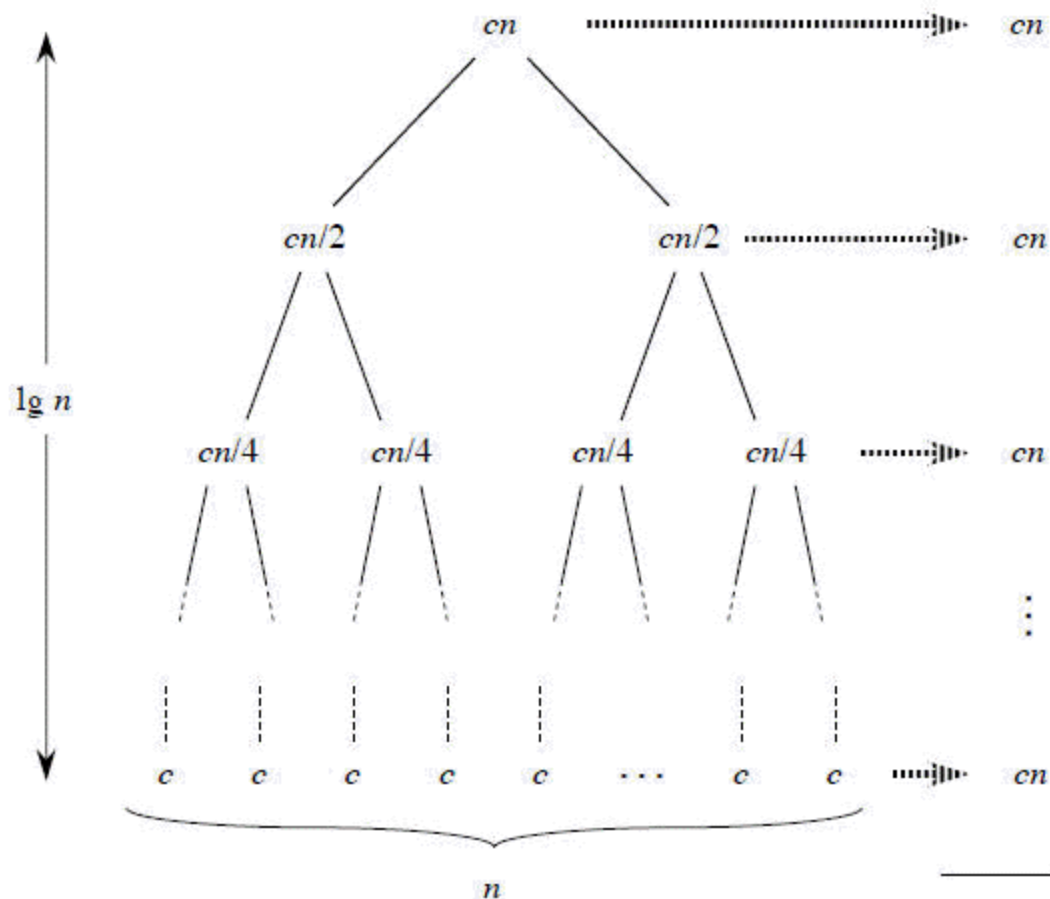
Modalità dell' esame

- Prova scritta di Algoritmi e Strutture Dati I ovvero Due prove intercorso (validità quattro appelli)
- Svolgimento di un progetto Algoritmi e Strutture Dati II da scegliersi tra una lista di progetti che verrà comunicata dal docente (validità quattro appelli).
- Superato lo scritto di ASD-I ed il progetto di ASD-II si accede all' Esame Orale congiunto di ASD-I e ASD-II

Testi Consigliati

- ALGORITMI:
- Cormen, Leiserson, Rivest, Stein
*Introduzione agli Algoritmi e
alle strutture Dati*
(seconda edizione o terza edizione)
Ed. Mc Graw-Hill.

Merge Sort : Albero di Ricorsione



(cont.)

- Sommiamo i costi per ogni livello dell' albero.
- Il primo livello in alto ha un costo totale cn ; il secondo livello ha costo $\frac{cn}{2} + \frac{cn}{2} = cn$; il terzo livello ha costo $\frac{cn}{4} + \frac{cn}{4} + \frac{cn}{4} + \frac{cn}{4} = cn$; e così via.
- In generale, il livello i ha 2^i nodi, ciascuno con costo $c \frac{n}{2^i}$, quindi il costo totale del livello è $2^i c \frac{n}{2^i} = cn$. All' ultimo livello in basso ci sono n nodi, ciascuno con costo c , per un costo totale di cn .

(cont.)

- Il numero totale dei livelli è $\log_2 n + 1$, dove n è la dimensione dell' input.
- Per dimostrarlo, si utilizza il metodo induttivo. Il caso base si verifica quando $n = 1$, quando c' è un solo livello, si ha che il numero di livelli è 1.
- Supponiamo per l' ipotesi induttiva che il numero di livelli con 2^i nodi per l' albero di ricorsione sia $\log_2 2^i + 1 = i + 1$. Un albero con 2^{i+1} nodi ha un livello in più di un albero con 2^i nodi, quindi il numero dei livelli è $(i + 1) + 1 = \log_2 2^{i+1} + 1$.

(cont.)

- Per calcolare il costo totale della ricorrenza, basta sommare i costi di tutti i livelli. Ci sono $\log_2 n + 1$ livelli, ciascuno di costo cn , per un costo totale di $cn(\log_2 n + 1)$.
- Ignorando la costante ed il termine di ordine inferior, abbiamo che la complessità del MERGE SORT è $\Theta(n \log_2 n)$.
- **E tale risultato vale sia nel caso peggiore, nel caso favorevole e nel caso medio.**

Serie Numeriche (da ricordare)

- Serie Aritmetica $\sum_{i=1}^n i = \frac{1}{2}n(n+1) = \Theta(n^2)$
- Sommatoria dei Quadrati
$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$
- Sommatoria dei Cubi $\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4} = \Theta(n^4)$

(cont.)

- Serie armoniche $\sum_{k=1}^n \frac{1}{k} = \log_e n + O(1)$
- Serie geometriche $\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$
- Serie geometriche infinite $\sum_{k=1}^{\infty} x^k = \frac{1}{1-x}$
- Serie geometriche $\sum_{k=1}^n kx^k = \frac{x}{(1-x)^2}$

Ricorrenze

Risoluzione per Ricorsione

- Nella Tecnica Divide-et-impera i problemi vengono risolti in maniera ricorsiva applicando i passi Divide, Impera e Combina ad ogni livello di ricorsione.
- Quando i sottoproblemi sono abbastanza grandi da essere risolti ricorsivamente, si ha il **caso ricorsivo**.
- Una volta che i sottoproblemi diventano sufficientemente piccoli da non richiedere ricorsione, si è raggiunto il **caso base**.

Ricorrenza

- Una ricorrenza è un' equazione o disequazione che descrive in termini del suo valore con input più piccoli.

Derivazione della ricorrenza per la complessità di un algoritmo

- Per derivare le relazioni di ricorrenza per la complessità di un algoritmo $T(n)$ è necessario:
 - determinare le dimensioni di un input;
 - determinare quale valore di n , indicato con n_0 , viene utilizzato per il caso base della ricorsione;
 - Determinare la complessità $T(n_0)$ del caso base della ricorsione (solitamente è una costante $O(1)$).

Esempio

- $$T(n) = \begin{cases} c & \text{se } n = n_0 \\ aT(f(n)) + g(n) & \text{se } n > n_0 \end{cases}$$
 - n_0 è la base della ricorsione;
 - c è la complessità per la base della ricorsione;
 - a è il numero delle chiamate ricorsive effettuate;
 - $f(n)$ è la dimensione dell' input dei problemi risolti con le chiamate ricorsive;
 - $g(n)$ è la complessità di calcolo dell' algoritmo non generate dalle chiamate ricorsive.

Metodi di Risoluzione delle Ricorrenze

- Metodo di Sostituzione
- Metodo dell' albero di ricorsione
- Metodo del teorema dell' esperto
- (Metodo iterativo)

Metodo di sostituzione

- Nel metodo di sostituzione, ipotizziamo un limite e poi usiamo l' induzione matematica per dimostrare che la nostra ipotesi è corretta.

Metodo dell' albero di Ricorsione

- Il metodo dell' albero di ricorsione converte la ricorrenza in un albero i cui nodi rappresentano i costi ai vari livelli della ricorsione; per risolvere la ricorrenza, adotteremo delle tecniche che limitano le sommatorie.

Il metodo dell' esperto (Master Theorem)

- Il metodo dell' esperto fornisce i limiti per ricorrenze nella forma $T(n) = aT\left(\frac{n}{b}\right) + f(n)$
 - dove $a \geq 1, b > 1$ e $f(n)$ è una funzione data.

Metodo Iterativo

- Iteriamo la ricorrenza (senza costruire l' albero di ricorrenza) fino a calcolare la soluzione.

Esempio

$$T(n) = T(n - 1) + \Theta(1)$$

$$T(n) = T(n - 2) + \Theta(1) + \Theta(1)$$

$$T(n) = T(n - 3) + \Theta(1) + \Theta(1) + \Theta(1)$$

....

$$T(n) = \sum_{i=1}^n \Theta(1) = n \Theta(1) = \Theta(n)$$

Esempio

$$T(n) = T(n - 1) + n^2$$

$$T(n) = T(n - 2) + n^2 + (n - 1)^2$$

$$T(n) = T(n - 3) + n^2 + (n - 1)^2 + (n - 2)^2$$

....

$$T(n) = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

Metodo di sostituzione

- Il metodo di sostituzione richiede i seguenti passi:
 - ipotizziamo la forma della soluzione;
 - usiamo l' induzione matematica per dimostrare che la nostra ipotesi è corretta.

Esempio di utilizzo del metodo della sostituzione

- Determinare un limite superiore per la ricorrenza
$$T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + n$$
- Soluzione:
 - Ipotizziamo la soluzione.
 - Questa ricorrenza è analoga a qualcuna incontrata in precedenza ?

Ipotesi della Soluzione

- La ricorrenza ci ricorda quella del merge sort. Pertanto è naturale ipotizzare una soluzione $T(n) = O(n \log_2 n)$.
- Ipotizziamo che sia vera per $\frac{n}{3}$ allora abbiamo:
$$T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) \leq c \left\lfloor \frac{n}{3} \right\rfloor \log_2 \left(\left\lfloor \frac{n}{3} \right\rfloor\right)$$

Sostituzione

- Sostituendo abbiamo:
- $T(n) \leq 3c \left\lfloor \frac{n}{3} \right\rfloor \log_2 \left(\left\lfloor \frac{n}{3} \right\rfloor \right) + n$
- $T(n) \leq cn \log_2 \frac{n}{3} + n$
- $T(n) \leq cn \log_2 n - cn \log_2 3 + n$
- $T(n) \leq cn \log_2 n - n(c \log_2 3 - 1)$
- $T(n) \leq cn \log_2 n \quad \left(\text{per } c \geq \frac{1}{\log_2 3} \right)$

MDS: Formulazione di una buona ipotesi

- Non esiste un metodo generale per indovinare la soluzione corretta di una ricorrenza.
- Se una ricorrenza è simile a una vista in passato, allora ha senso utilizzare una soluzione analoga.

Esempio

- La ricorrenza $T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor + 17\right) + n$ sembra complicata per la presenza del termine 17.
- In realtà non influisce sulla soluzione della ricorrenza. Quando n è grande, $T\left(\left\lfloor \frac{n}{3} \right\rfloor + 17\right)$ si comporta come $T\left(\left\lfloor \frac{n}{3} \right\rfloor\right)$, pertanto facciamo l'ipotesi che $T(n) = O(n \log_2 n)$.
- Sostituendo l'ipotesi è verificata (otteniamo la stessa formula del caso precedente).

MDS: Finezze

- Ci sono casi in cui è possibile ipotizzare correttamente un limite asintotico per la soluzione di una ricorrenza, ma in qualche modo i calcoli matematici non tornano quando operiamo la sostituzione.
- Normalmente, l'ipotesi induttiva non è abbastanza forte per dimostrare il limite esatto. In tal caso basta correggere l'ipotesi sottraendo un termine di ordine inferiore per verificare l'ipotesi.

Esempio

- $T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + 1$
- Supponiamo che la soluzione sia $T(n) = O(n)$.
Proviamo a dimostrare che $T(n) \leq cn$, abbiamo:
$$T(n) \leq c \left\lfloor \frac{n}{2} \right\rfloor + c \left\lceil \frac{n}{2} \right\rceil + 1 \leq cn + 1$$
 che non implica $T(n) \leq cn$ qualunque sia il valore di c .
- La nostra ipotesi non funziona per la presenza della costante 1. Superiamo la difficoltà sottraendo un termine di ordine inferiore alla precedente.

(cont.)

- La nuova ipotesi è $T(n) \leq cn - d$, dove $d \geq 0$.
Sostituendo

$$T(n) \leq \left(c \left\lfloor \frac{n}{2} \right\rfloor - d\right) + \left(c \left\lfloor \frac{n}{2} \right\rfloor - d\right) + 1$$

$$T(n) \leq cn - 2d + 1 \leq cn - d \quad \text{per ogni } d \geq 1.$$

MDS: Tranelli

- Consideriamo la $T(n) = 3T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + n$
- Potremmo dimostrare erroneamente che $T(n) = O(n)$, supponendo che $T(n) \leq cn$ deducendo che:

$T(n) \leq 3\left(c\left\lfloor \frac{n}{3} \right\rfloor\right) + n \leq cn + n = O(n)$, in quanto c è costante. L'errore sta nel fatto che noi non abbiamo dimostrato la forma esatta dell'ipotesi, ossia, $T(n) \leq cn$.